RAG_thread

Retrieval-Augmented Question Answering over Email Threads using Wikipedia Text and Tables on Indian States and Union Territories

Smrithi
M.Sc Integrated Data Science, PSG College of Technology, Coimbatore
Take-home assignment, AI Engineering Intern, Intel | October 2026

Abstract

Email threads hide the real question under quoted replies, signatures and references such as “that state” or “the larger one”, and the answer is often inside a table rather than in prose. RAG_thread reads an email thread or plain text, cleans it, uses an LLM to rewrite it into standalone questions, and answers them from 71 Wikipedia pages on the Indian states and union territories. The knowledge base contains 7,470 section-aware chunks of text, table rows converted to sentences, infobox fields and image captions. Retrieval combines a dense bi-encoder (bge-small) over a NumPy matrix with a hand-written BM25, fuses the two rankings with Reciprocal Rank Fusion, diversifies them with Maximal Marginal Relevance and re-ranks the top candidates with a cross-encoder. A Gemini model then writes a short answer that cites the retrieved passages, or reports “not found” through two independent gates. Near-duplicate text is removed with MinHash and LSH, implemented from scratch. On 25 test threads the final re-ranked stage reaches Recall@5 = 0.92, MRR@10 = 0.775 and nDCG@5 = 0.640. The system makes the correct answer/not-found decision on 24 of 25 threads, and every answer it gives is correct, cited and numerically grounded.

1. Problem Statement

Support and operations staff often receive long email threads in which the actual question is buried. The same text is quoted several times, signatures and legal disclaimers add noise, and the latest message refers back to earlier ones (“which of the two is larger?”, “what is the capital of that state?”). Staff then search reference material by hand, and the answer is frequently in a table (an infobox field, a census table, a list of districts), not in running text.

Goal: build a system that (1) reads an email thread or plain text, (2) works out what is actually being asked and turns it into self-contained questions, (3) retrieves supporting text and table rows from Wikipedia pages on the Indian states and union territories, and (4) returns a short answer grounded in, and citing, the retrieved passages, or says “not found” when the pages do not contain the answer.

1.1 Requirements derived from the task

Requirement	How it is met
Input is an email thread or plain text	Thread parser handles Gmail-style (On … wrote:, > quotes) and Outlook-style (-----Original Message-----, From:/Sent:) threads; plain text is treated as one message
Understand what the thread is asking	Thread cleaning + LLM rewrite into standalone questions with references resolved
Retrieve from Wikipedia text and tables	Section-aware text chunks + every table/infobox row as a sentence chunk; captions kept for images

Requirement	How it is met
LLM prints the answer	Gemini generates an answer only from the top-5 passages, with [n] citations
Scale with semantic search and a vector store	Dense embeddings in a NumPy vector store + BM25 hybrid, fusion, MMR and re-ranking
Different from my earlier Multimodal QA project	Different input, data, retrieval architecture and evaluation; see Section 6

2. Assumptions

- **Thread format:** threads are pasted as plain text, newest message first (top-posting), as most mail clients do. Reply markers follow common Gmail and Outlook conventions.
- **Question location:** the open question is usually in the latest message. If the latest message only follows up ("any update?"), the earlier unanswered question is the target.
- **Scope:** answers come only from the Wikipedia pages in the knowledge base. Questions outside this scope (sport results, other countries, forecasts) must be answered with "not found", not with the LLM's own knowledge.
- **Language:** threads and pages are in English. Indian state short forms (UP, TN, Pondy, Orissa) are common in real mail and must be understood.
- **Freshness:** a snapshot of Wikipedia (October 2026) is acceptable. Facts are reported with the year stated in the source, since figures differ between census years.
- **Images:** only image captions are used (the task marks images as optional), so no vision model is needed.
- **Deployment:** the system runs on a normal laptop CPU. The only external service is the Gemini API (free tier), whose requests-per-minute limits must be respected.

3. Dataset and Why It Was Chosen

The knowledge base is the English Wikipedia article for each of the **28 states and 8 union territories**, the summary page **"States and union territories of India"** (one comparison table row per state), and the **"List of districts of ..."** page for each state or UT. That gives 73 titles, of which 71 exist (Chandigarh and Lakshadweep are single-district UTs without a list page).

3.1 Why this dataset

In one line: the Indian states and union territories are a closed set of 36 entities with parallel, table-heavy Wikipedia pages, which is exactly how regions appear in Indian business email (abbreviations, comparisons, "that state"), so the domain is realistic, makes retrieval hard, and keeps every answer checkable.

- **Rich in tables:** every state page has an infobox (capital, area, population, literacy, languages, GDP) plus census and demographic tables, and the district pages are almost entirely tables. That tests the "answer is in a table" part of the problem directly.
- **Natural email-thread questions:** comparisons between states, references such as "that state", and abbreviations (UP, TN) occur naturally in Indian business mail, so realistic test threads are easy to write.
- **Bounded and checkable:** about 71 pages is large enough to need real retrieval (7,470 chunks) and small enough to verify answers by hand and keep the index in memory.

- **Hard negatives built in:** 36 pages share the same structure (“Capital”, “Official language”, “Districts”), so a naive retriever confuses states. This motivates the entity filter, hybrid retrieval and re-ranking.
- **Open licence:** Wikipedia text is available under CC BY-SA, and every answer links back to the source page and section.

3.2 Dataset statistics

Item	Count
Pages loaded	71 (37 main/summary + 34 district lists)
Sections kept	1,509
Tables (wikitables + infoboxes)	258
Image captions	968
Chunks after chunking and dedup	7,470 (2,522 text, 4,319 table rows, 629 caption groups)
Average chunk length	text 120.7 words, table row 20.8 words, captions 14.6 words
Near-duplicate chunks removed (MinHash)	8
BM25 vocabulary / average document length	29,938 terms / 45.7 tokens
Embedding matrix	7,470 × 384 float32 (≈11 MB)

Bibliographic sections (References, See also, External links, Notes, Further reading) are excluded. They contain source titles rather than facts, and they add many query-like words (“Census”, “Literacy”, “2011”) that would pollute BM25 results.

4. Proposed Solution

The system has an **offline** stage that builds the knowledge base once and an **online** stage that answers each thread (Figure 1).

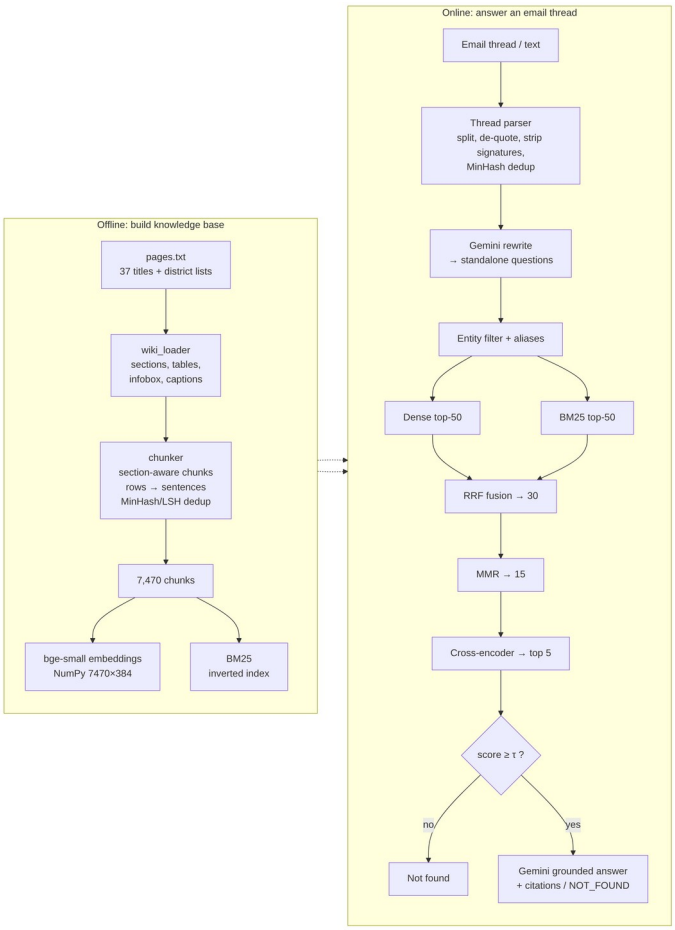


Figure 1. System architecture: offline knowledge-base construction (left) and online question answering (right).

4.1 Online pipeline, step by step

1. **Parse the thread:** split it into messages at reply markers, strip > quote marks, remove headers, greetings, sign-offs, signatures and disclaimers, order the messages oldest-first, and drop repeated paragraphs with MinHash.
2. **Rewrite (Gemini):** from the cleaned message list (the thread memory), produce one standalone question per item asked. References and abbreviations are resolved and years are kept.
3. **Entity filter:** detect state/UT names and aliases in each question and restrict the search to those states' pages plus the summary page.
4. **Hybrid retrieval:** take the dense cosine top-50 and the BM25 top-50.
5. **Fusion and diversity:** RRF over the two rankings (keep 30), then MMR selects 15 relevant but non-redundant candidates.
6. **Re-rank:** a cross-encoder scores each (question, chunk) pair and the top 5 are kept.
7. **Gate 1, retrieval:** if the best re-rank probability is below $\tau = 0.2$, return "not found" without calling the LLM.
8. **Grounded answer (Gemini):** answer only from the 5 numbered passages, cite each fact as [n], and reply NOT_FOUND if the passages do not contain the answer (Gate 2).
9. **Present:** the Streamlit UI and the CLI show the cleaned thread, the questions, every retrieval stage, the answer and the clickable sources.

5. Approach and Design Decisions

The project was built in phases, each checked before moving on: thread parser and loader, chunker, index, retrieval, LLM integration, interface, evaluation. Each decision below was made for a reason specific to this problem, and several were confirmed by measurements during development.

Decision	Alternatives considered	Why this choice
De-quote, then dedup quoted text (not delete it)	Delete all > lines	In a pasted thread the quote is often the only copy of an older message. Deleting it would lose the context needed to resolve "that state". MinHash removes only the repeated copies.
MinHash + LSH for dedup	Exact string match; pairwise Jaccard	Quoted copies differ slightly (line wraps, prefixes), so exact match fails. LSH avoids comparing every pair, $O(n^2)$.
Section-aware chunks with page | section prefix	Fixed-size sliding windows	Each chunk keeps one topic and can be cited as page § section. The prefix gives table rows such as "Literacy: 95.3%" the state they belong to.
Each table row as one sentence chunk	Whole table as one chunk; table QA model	One fact per chunk is easy to retrieve and cite, and the same retriever handles text and tables.
No MinHash on table rows	Dedup everything	Rows of one table share the prefix and column names, so their Jaccard is high even when the values differ. MinHash would delete real records.
bge-small-en-v1.5 embeddings	Larger models; OpenAI embeddings	Strong retrieval quality for its size (384-d); runs on CPU in about 1 minute for 7,470 chunks; free and local.
Own NumPy vector store	FAISS, Chroma	At 7.5k vectors an exact search is one matrix–vector product (about 1 ms). An ANN index adds a dependency without any speed benefit at this size.
Hand-written BM25 alongside dense	Dense only	Measured: dense missed "God's Own Country" and ranked "Country: India" above the capital row. BM25 found both. Names and numbers need lexical matching.
RRF fusion	Weighted score sum	Cosine (0–1) and BM25 (0–30+) scores are on different scales. RRF uses only ranks, so it needs no score normalisation or weight tuning.
MMR before re-ranking	Top-k only	Comparison questions need evidence for both states. MMR stops the pool being filled with near-identical chunks about one state.
Cross-encoder re-ranking of 15 chunks	No re-rank; LLM re-rank	Reads the question and chunk together, which is far more accurate than the bi-encoder. Limiting it to 15 pairs keeps latency at about 0.1 s.
Entity filter with an alias file	NER model; LLM extraction	36 look-alike pages make cross-state confusion the main error. The filter is cheap and transparent, and the rewrite step already expands UP → Uttar Pradesh.
Gemini for rewrite and answer	Local LLM	Needs reasoning over the whole thread; free tier; the model name is configurable, with an automatic fallback model.
Two not-found gates	Threshold only	Measured: "Kerala's GDP in 2040" retrieves Kerala GDP chunks with score 0.9986. Only the LLM gate rejects it.
Temperature 0	Default sampling	Repeatable, less creative answers for a factual task.

6. How This Differs from My Earlier Multimodal QA Project

My earlier project was a Flask web app that answered questions about one uploaded PDF or image. It extracted text with PyMuPDF and OCR, captioned images with ViT-GPT2, embedded questions with BERT (projected 768 → 384 by a linear adapter) and chunks with all-MiniLM-L6-v2, retrieved the top chunks by cosine similarity, generated answers with OPT-1.3B, and supported voice input and output. The task asked for a RAG system that is similar in spirit but different in architecture and approach. RAG_thread keeps the core idea (retrieve, then generate) and changes almost everything around it. It also fixes the weaknesses I identified in the earlier system.

Aspect	Earlier Multimodal QA	RAG_thread (this project)
Problem	Ask a direct question about one uploaded document	Work out the real question hidden inside a noisy, multi-reply email thread, then answer it from a fixed knowledge base
Input	Uploaded PDF / image + typed or spoken question	Email thread (Gmail/Outlook style) or plain text; no voice
Input understanding	None; the question is used as typed	Thread parser (de-quote, headers, signatures, disclaimers, MinHash dedup) + LLM rewrite that resolves references and abbreviations and splits multi-question mails
Knowledge source	Whatever the user uploads, processed per upload	71 Wikipedia pages built offline into a persistent index (data committed to the repository)
Modalities	Text + images (ResNet50 features, ViT-GPT2 captions, OCR)	Text + tables + infobox fields; images only through captions (task makes images optional)
Tables	Treated as plain page text	Every table and infobox row becomes its own citable sentence chunk with page | section | caption context
Chunking	Fixed 512-word sliding window	Section-aware chunks (≤ 200 words, 30 overlap) that never cross a section, plus row-level table chunks
Embeddings	BERT for questions vs MiniLM for chunks, bridged by an untrained linear adapter	One model (bge-small) for both, with its query instruction, so questions and chunks share one embedding space
Retrieval	Dense cosine only, top-5	Entity filter → dense + hand-written BM25 → RRF → MMR → cross-encoder re-rank
Re-ranking	Re-scored against the first chunk's embedding (a known flaw)	Cross-encoder that reads the question and the chunk together
Duplicates	Not handled	MinHash + LSH (from scratch) for quoted text and chunks; MMR for semantic duplicates
Generator	OPT-1.3B run locally (≈8 GB RAM), temperature 0.3 with sampling	Gemini via API, temperature 0, with an automatic fallback model and an evidence-only fallback
Grounding and citations	Prompt says "use only the context"; no citations	Every fact cited as [n] and linked to the Wikipedia page and section; conflicting values reported with their year
Unanswerable questions	Relies on the LLM saying "not enough information"	Two gates: re-rank score threshold (no LLM call) + LLM NOT_FOUND
Model loading	Reloaded per request (slow)	Loaded once and cached (Streamlit cache_resource)
Interface	Flask REST API + HTML/Bootstrap, voice I/O	Streamlit app that shows every pipeline stage, plus a CLI

Aspect	Earlier Multimodal QA	RAG_thread (this project)
Evaluation	Substring match of the expected answer	25 labelled threads; Recall@5, MRR, nDCG@5 for every stage; not-found accuracy with a threshold sweep; citation validity and number grounding

6.1 Lessons carried over

- **Shared embedding space:** the untrained 768 → 384 adapter in the earlier project meant question and chunk vectors were not truly comparable. Here a single embedding model encodes both.
- **Re-rank against the question:** the earlier re-ranking did not use the question at all. Here a cross-encoder scores each (question, chunk) pair directly.
- **Keyword plus semantic:** the earlier project listed "BM25 + semantic hybrid" as a future improvement. It is now implemented, and the evaluation shows why it matters (Section 9.1).
- **Load once:** models are loaded once per server instead of per request.
- **Measure, don't assume:** the earlier substring-accuracy check is replaced by standard IR metrics per stage, so every component's contribution is visible.

7. Algorithms and Mathematical Formulation

Equations are numbered, and every symbol is defined directly below the equation that introduces it.

7.1 Thread cleaning

Lines are scanned in order. Leading > characters are counted (quote depth) and removed. A line that matches a reply marker ("On … wrote:", "-----Original Message-----", "From:") starts a new message, and the sender's name is taken from it. Within each message, header lines are removed (the Subject is kept), greeting lines are dropped, and everything from a sign-off line ("Regards,", "Thanks,"), "--" or a disclaimer onwards is cut. The name after the sign-off becomes the sender when no header gives one. Messages are reversed to chronological order, and repeated paragraphs (at least 5 words) are removed with MinHash (Section 7.2), keeping the first occurrence.

7.2 Shingling, Jaccard similarity, MinHash and LSH

Each text is turned into its set of word 3-shingles (every run of 3 consecutive words). Two texts are compared with the Jaccard index:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \tag{1}$$

where

$A,\ B$	sets of word 3-shingles of the two texts
$\lvert \cdot \rvert$	number of elements in a set
$\cap,\ \cup$	set intersection and union
$J(A, B)$	Jaccard similarity, between 0 (nothing shared) and 1 (identical sets)

Each shingle is mapped to an integer and hashed with n = 64 universal hash functions. The MinHash signature of a set keeps the smallest hash value under each function:

$$h_i(x) = (a_i\, x + b_i) \bmod p \tag{2}$$

$$\mathrm{sig}_i(A) = \min_{x \in A} h_i(x) \tag{3}$$

where

x	integer id of a shingle (first 60 bits of its MD5 hash, stable across runs)
$i,\ n$	index of the hash function, i = 1, …, n, with n = 64 hash functions
$a_i,\ b_i$	random integers, $a_i \in [1, p-1]$ and $b_i \in [0, p-1]$, drawn once with a fixed seed
p	the Mersenne prime $2^{61} - 1$
$h_i(x)$	value of the i-th hash function for shingle x
$\mathrm{sig}_i(A)$	i-th value of the MinHash signature of set A

For a random hash function, the minimum over A ∪ B is equally likely to be any of its elements, and the two signatures agree exactly when that element is in A ∩ B. So the fraction of matching positions estimates J:

$$P\left[\mathrm{sig}_i(A) = \mathrm{sig}_i(B)\right] = J(A, B) \tag{4}$$

$$\hat{J}(A, B) = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}\left[\mathrm{sig}_i(A) = \mathrm{sig}_i(B)\right] \tag{5}$$

where

$P[\cdot]$	probability over the random choice of hash function
$\hat{J}(A, B)$	MinHash estimate of J(A, B); a unit test gives 0.602 for a true J of 0.588
$\mathbf{1}[\cdot]$	indicator: 1 if the condition holds, otherwise 0

LSH banding. The signature is split into b bands of r rows, and two texts become candidates if any whole band is identical:

$$P(\text{candidate}) = 1 - (1 - s^r)^b\,, \qquad t \approx \left(\frac{1}{b}\right)^{1/r} = \left(\frac{1}{8}\right)^{1/8} \approx 0.77 \tag{6}$$

where

s	true Jaccard similarity of the two texts
b	number of bands (8)
r	rows, i.e. signature values, per band (8), so that b · r = n
t	similarity at which a pair becomes a candidate with about 50% probability

Candidates are confirmed when the estimate is at least 0.8. Pairs well below t almost never share a band, so the cost is close to linear in the number of texts instead of quadratic.

7.3 Chunking

Paragraphs within one section are packed together up to 200 words. Longer paragraphs are cut into 200-word windows with a 30-word overlap, so a fact split at a boundary appears whole in at least one window. Every table and infobox row becomes “column: value, column: value” (empty, repeated-header and numeric-index columns are skipped), and every chunk’s searchable text is “page | section [| caption] | body”.

7.4 Dense retrieval: embeddings and cosine similarity

The bi-encoder bge-small-en-v1.5 embeds every chunk once. A question is embedded with the instruction prefix the model was trained with (asymmetric retrieval). Similarity is the cosine of the angle between the two vectors:

$$\cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \, \|\mathbf{d}\|} = \frac{\sum_{j=1}^{D} q_j \, d_j}{\sqrt{\sum_{j=1}^{D} q_j^2} \, \sqrt{\sum_{j=1}^{D} d_j^2}} \tag{7}$$

All vectors are L2-normalised (unit length), so the cosine equals the dot product, and all chunks are scored with one matrix–vector product (exact nearest-neighbour search):

$$\mathbf{s} = E \, \mathbf{q} \tag{8}$$

where

$\mathbf{q}$	embedding of the question
$\mathbf{d}$	embedding of a chunk
D	embedding dimension (384)
$q_j,\ d_j$	j-th components of q and d
$\|\cdot\|$	Euclidean (L2) norm
E	N × D matrix whose rows are the normalised embeddings of all N = 7,470 chunks
$\mathbf{s}$	vector of N cosine scores, one per chunk

7.5 BM25

Tokens are lowercase alphanumeric words with stop words removed. The lexical score of chunk d for question q is:

$$\mathrm{BM25}(q, d) = \sum_{t \in q} \mathrm{IDF}(t) \cdot \frac{f(t, d) \, (k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \, \dfrac{|d|}{\mathrm{avgdl}} \right)} \tag{9}$$

$$\mathrm{IDF}(t) = \ln\left(\frac{N - n(t) + 0.5}{n(t) + 0.5} + 1 \right) \tag{10}$$

where

$q,\ d$	the question (as a list of tokens) and a chunk
t	a term (token) of the question
$f(t, d)$	number of times t occurs in chunk d
$\|d\|$	length of chunk d in tokens
avgdl	average chunk length (45.7 tokens)
N	number of chunks (7,470)
$n(t)$	number of chunks that contain t
k_1	term-frequency saturation (1.5): repeats of a term add less and less
b	length normalisation (0.75); unrelated to the LSH bands of Section 7.2
$\ln$	natural logarithm

IDF gives rare terms (e.g. “Thiruvananthapuram”) more weight than common ones. An inverted index (term → list of [chunk, f(t, d)]) means a query only touches chunks that contain its terms.

7.6 Entity filter

State/UT names (from the page list) and aliases (UP, TN, Orissa, Pondy, …) are matched as whole words, longest names first. Aliases written in capitals are matched case-sensitively, so “set up” is not read as Uttar Pradesh. If any entity is found, only chunks from pages whose title contains it, plus the summary page, are kept in both rankings.

7.7 Reciprocal Rank Fusion

$$\mathrm{RRF}(d) = \sum_{m \in M} \frac{1}{k + \mathrm{rank}_m(d)} \tag{11}$$

where

d	a chunk
M	the rankings being fused: {dense, BM25}
$\mathrm{rank}_m(d)$	1-based position of d in ranking m; the term is omitted if d is not in that ranking’s top 50
k	smoothing constant (60)

RRF uses only ranks, so the different score scales of cosine (0–1) and BM25 (0–30+) do not matter. The constant k flattens the curve, so a chunk ranked fairly well by both methods beats one ranked first by only one of them.

7.8 Maximal Marginal Relevance

$$d^* = \arg\max_{d \in R \setminus S} \left[\lambda \, \mathrm{sim}(\mathbf{q}, d) - (1 - \lambda) \max_{d' \in S} \mathrm{sim}(d, d') \right] \tag{12}$$

where

$\mathbf{q}$	question embedding
R	the 30 candidates kept after RRF
S	chunks selected so far (starts empty and grows to 15)
$R \setminus S$	candidates not yet selected
d^*	the next chunk added to S
$\mathrm{sim}(\cdot, \cdot)$	cosine similarity of embeddings, Eq. (7)
λ	weight between relevance and diversity (0.7)

Each step adds the candidate that best balances relevance to the question against similarity to chunks already chosen. This handles semantic duplicates, where MinHash handled lexical ones.

7.9 Cross-encoder re-ranking (attention and sigmoid)

The cross-encoder (MiniLM-L6, trained on MS MARCO) reads “[CLS] question [SEP] chunk” as one sequence. In every layer, self-attention lets each question token attend to each chunk token:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \qquad \text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_j e^{z_j}} \tag{13}$$

where

$Q,\ K,\ V$	query, key and value matrices: the token representations of the input multiplied by learned weight matrices (one row per token)
$K^\top$	transpose of K
d_k	dimension of each key vector; dividing by its square root keeps dot products in a stable range
$\mathbf{z},\ z_i$	a vector of scores and its i-th element; softmax turns scores into weights that sum to 1
e	Euler's number (exponential function)

A linear layer on the [CLS] token gives a relevance logit z for the (question, chunk) pair, which the sigmoid turns into a probability:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{14}$$

where

z	relevance logit of the (question, chunk) pair
$\sigma(z)$	relevance probability between 0 and 1, used to rank the 15 MMR candidates and keep the top 5

Reading both texts together is much more accurate than comparing two separate vectors, but it costs one transformer pass per pair. That is why it runs only on the 15 MMR candidates.

7.10 Answer generation and the not-found gates

The LLM generates each token by sampling from a softmax with temperature over its vocabulary:

$$p_i = \frac{\exp(z_i/T)}{\sum_{j \in \mathcal{V}} \exp(z_j/T)} \tag{15}$$

where

$\mathcal{V}$	the LLM's vocabulary
z_i	the LLM's logit (unnormalised score) for vocabulary token i
T	temperature; as T → 0 the model always picks the most likely token (greedy decoding)
p_i	probability of generating token i next

T = 0 is used for repeatable factual answers. The prompt contains only the 5 numbered passages and rules: use only these facts, cite [n] after each fact, report conflicting values with their year, and reply NOT_FOUND if the answer is not present. Two gates decide whether an answer is given:

$$\text{Gate 1: answer only if} \quad \max_{d \in F} \sigma(z_d) \ \geq \ \tau \tag{16}$$

where

F	the final top-5 chunks after re-ranking
z_d	cross-encoder logit for the pair (question, d)
τ	not-found threshold (0.2); below it, "not found" is returned without calling the LLM

- **Gate 2 (generation):** the LLM replies NOT_FOUND when relevant-looking passages do not actually answer the question (e.g. a forecast for 2040).

7.11 Evaluation metrics

For each gold item, a chunk is relevant if it comes from one of the gold pages and contains all gold keywords.

$$\text{Recall@}k = \frac{1}{|G|}\sum_{g\in G}\mathbf{1}\left[r_g \le k\right], \qquad \text{MRR} = \frac{1}{|G|}\sum_{g\in G}\frac{1}{r_g} \tag{17}$$

$$\text{DCG@}k = \sum_{i=1}^{k}\frac{\text{rel}_i}{\log_2(i+1)}, \qquad \text{IDCG@}k = \sum_{i=1}^{\min(R_g,k)}\frac{1}{\log_2(i+1)}, \qquad \text{nDCG@}k = \frac{\text{DCG@}k}{\text{IDCG@}k} \tag{18}$$

where

G	set of gold items (25 in the test set)
g	one gold item: gold pages plus required keywords
r_g	rank of the first relevant chunk for g; its reciprocal counts as 0 if the rank is above 10
k	cut-off rank: 5 for Recall and nDCG, 10 for MRR
rel_i	1 if the chunk at rank i is relevant to g, otherwise 0
R_g	number of chunks in the whole corpus relevant to g
$\text{IDCG@}k$	DCG of an ideal ranking, so that nDCG@k lies between 0 and 1
$\mathbf{1}[\cdot]$	indicator: 1 if the condition holds, otherwise 0

Faithfulness (basic, automatic): (a) citation validity: every [n] refers to an existing passage; (b) citation coverage: each answer cites at least one passage; (c) number grounding, Eq. (19); (d) answer accuracy: the expected answer keywords appear in the answer.

$$\text{Number grounding} = \frac{\text{numbers in answers that also appear in the cited passages}}{\text{all numbers in answers}} \tag{19}$$

8. Implementation

The system is written in Python. Libraries are used only for the embedding model and cross-encoder (sentence-transformers), the Gemini API (google-genai), HTML fetching and parsing (requests, BeautifulSoup, pandas.read_html), cosine similarity (scikit-learn) and the interface (Streamlit). The thread cleaner, chunker, shingling/MinHash/LSH, BM25, entity filter, RRF, MMR, thresholding and every evaluation metric are implemented from scratch.

Module	Responsibility
src/parse_thread.py	Split, de-quote, strip headers/greetings/signatures, chronological order, MinHash paragraph dedup
src/minhash.py	Shingles, universal hashing, signatures, Jaccard, LSH banding, duplicate detection
src/wiki_loader.py	Wikipedia API download (retry on 429, polite delays), section/table/infobox/caption parsing, district-page expansion
src/chunker.py	Section-aware text windows, rows to sentences, caption groups, chunk dedup
src/index.py	bge-small embeddings, NumPy store, dense cosine search
src/bm25.py	Tokeniser, inverted index, IDF, BM25 scoring

Module	Responsibility
src/retrieve.py	Entity filter with aliases, RRF, MMR, cross-encoder re-rank, Gate 1
src/llm.py	Gemini client, main → fallback model, graceful failure
src/rewrite.py, src/answer.py	Prompt filling, JSON question list, grounded answer, citation extraction, Gate 2
src/pipeline.py, app.py, run_cli.py	End-to-end pipeline, Streamlit UI, CLI
src/evaluate.py	Runs the test set, stage-wise retrieval metrics, threshold sweep, faithfulness
prompts/*.txt, data/*.json	Prompts, aliases and test threads kept as data, separate from code

8.1 Key data structures

Parsed page (data/raw/<page>.json): `{title, url, sections: [{heading: "History > Colonial rule", text, tables: [{type: infobox|wikitable, caption, columns, rows}], captions}]}`.

Chunk (data/index/chunks.json): `{id, page, url, section, type: text|table|caption, body, text}`, for example text = `Kerala | Introduction | Capital: Thiruvananthapuram`.

Retrieval result: `{query, entities, dense, bm25, rrf, mmr, final, best_score, found}`. Every stage is a list of (chunk_id, score) and is kept for display and evaluation.

8.2 Robustness and practical engineering

- **API failures:** Wikipedia 429 errors trigger a wait-and-retry using the Retry-After header. Gemini errors (404/429/503) switch to a fallback model. If Gemini is down, rewriting falls back to the last message and the answer step shows the retrieved evidence instead of crashing.
- **Reproducibility:** fixed hash seed, temperature 0, cached Wikipedia data and a committed index. The index metadata check rejects an embedding file that does not match the chunks.
- **Latency:** models load once (about 37 s; cached with st.cache_resource in Streamlit). Retrieval takes 0.06–0.25 s per question. End-to-end time is about 2–4 s per question, mostly the Gemini calls.

9. Evaluation and Results

The test set has **25 hand-written threads**: easy (7), aliases (2), paraphrase (1), multi-turn references (2), follow-up only (1), noisy forwarded thread (1), two questions in one mail (2), numeric comparison (2), district tables (2), multi-hop (1) and unanswerable (4). Each case has gold pages and keywords and the expected answer keywords.

9.1 Retrieval by stage (ablation)

Stage	Recall@5	MRR@10	nDCG@5
Dense (bge-small)	0.880	0.667	0.547
BM25	0.760	0.693	0.477
RRF (dense + BM25)	0.880	0.743	0.553
MMR	0.880	0.673	0.529
Cross-encoder (final)	**0.920**	**0.775**	**0.640**

Fusion improves ranking quality over either retriever alone (MRR 0.743 vs 0.667 and 0.693). Re-ranking gives the best result on every metric (nDCG@5 0.553 → 0.640). MMR lowers MRR slightly, as expected, because it trades rank for diversity. Its purpose is to give the re-ranker evidence for both sides of a comparison, and the final stage then recovers and exceeds the RRF ordering. Final Recall@5 is 1.0 in every category except numeric comparison (0.75) and the single paraphrase case (0.0).

9.2 Answer / not-found decision

Outcome	Count
Answerable threads answered	20 / 21
Unanswerable threads correctly rejected	4 / 4
Decision accuracy	0.96

τ	0.01	0.05	0.1	0.2	0.3	0.5	0.7	0.9
Accuracy (score gate only)	0.96	0.96	0.96	0.92	0.92	0.92	0.92	0.92

The threshold sweep scores the retrieval gate alone. Low thresholds look slightly better only because they count the paraphrase miss as "answerable", even though the chunks retrieved for it were wrong. No threshold can reject "Kerala's GDP in 2040" (score 0.9986), which the LLM gate catches. τ = 0.2 was kept: it blocks clearly off-topic questions cheaply and leaves borderline cases to Gate 2.

9.3 Answer quality

Metric	Result
Answer accuracy (expected keywords present)	20 / 20 = 1.00
Answers with at least one citation	1.00
Invalid citations	0
Number grounding (numbers found in cited passages)	29 / 29 = 1.00

9.4 Error analysis

- **Fixed during evaluation (t03, official animal of the Andaman and Nicobar Islands):** the right row was ranked first, but the LLM said NOT_FOUND. The LLM context contained only the chunk body ("Animal, Dugong …"), without the table caption "Official symbols of …". Passing the full chunk text, including its prefix, to the LLM fixed it. Lesson: the context given to the LLM must keep the structure that gives a table row its meaning.
- **Remaining miss (t07, "Which Indian state is known as God's Own Country?"):** dense retrieval ranks the Kerala chunk low, and the extra word "Indian" pushed it out of the 30-candidate pool, so the re-ranker never saw it (best score 0.17). It was deliberately **not** fixed by enlarging the pool, because that would be tuning on the test set.
- **Caught by design (t24, Kerala's GDP in 2040):** retrieval correctly finds Kerala GDP chunks, and only the generation gate recognises that a forecast is not in the data.
- **Data conflicts handled:** Kerala's literacy appears as 94% (2011 census text), 93.9% and 95.3% (2024 infobox). The answer prompt makes the model report each value with its year instead of merging them.

Gemini outputs vary slightly between runs even at T = 0, and two rewrites in the final run were served by the fallback model after 429 errors. Retrieval metrics therefore vary by about ±0.01–0.04 between runs.

10. Screenshots

The Streamlit interface was tested with each of its three input modes, each on a different edge case. Every result shown was checked against Wikipedia. Case 1 also shows the ranking at every retrieval stage. More screenshots (parsed pages, chunks, CLI runs and evaluation output) are in the repository's screenshots/ folder.

10.1 Case 1: Sample thread (noisy multi-reply thread with references)

Edge case: Gmail quotes two levels deep, a duplicate Outlook copy of the first mail, signatures and a disclaimer. The last mail asks two questions using "the two" and "that state".



Figure 2. Case 1, input and processing: the raw thread (sidebar) is cleaned into three chronological messages (15 quoted lines, 11 signature lines and 1 duplicate paragraph removed) and rewritten into two standalone questions.

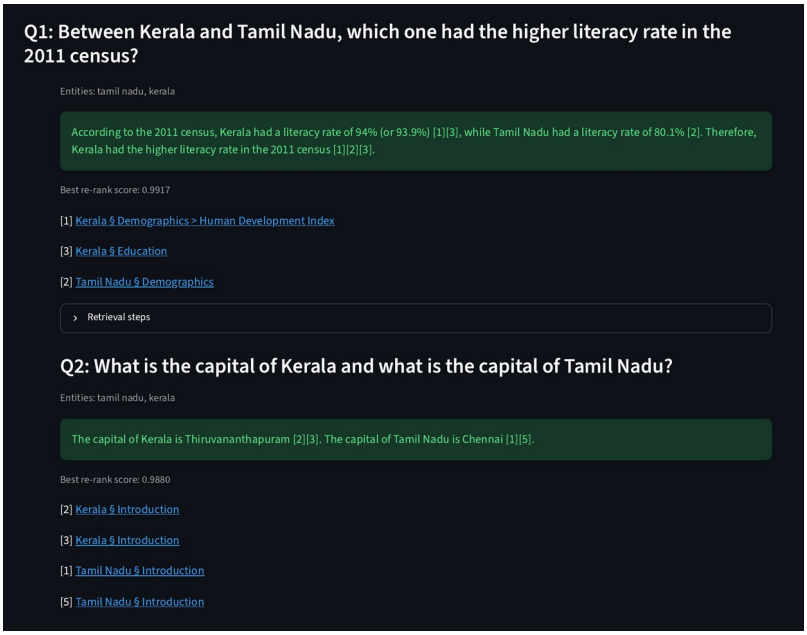


Figure 3. Case 1, output: Kerala had the higher 2011 literacy rate (94% vs 80.1%), and both capitals are given. Every fact is cited and linked to its Wikipedia section.



dense

rank	score	page	section	type	text
1	0.7704	Kerala	Introduction	table	Capital: Thiruvananthapuram
2	0.7671	Kerala	Introduction	text	Kerala (English: /ˈkɛrələ/ ⓘ/ KERR-ə-lə; Malayalam: [keːɾɐɭəm] ⓘ), officially Keralam, is a st
3	0.759	Kerala	Introduction	table	Country: India
4	0.7473	Tamil Nadu	Introduction	table	Capital and largest city: Chennai
5	0.7443	Tamil Nadu	Introduction	text	Tamil Nadu is the southernmost state of India. The tenth largest Indian state by area and th

bm25

rank	score	page	section	type	text
1	20.3144	Tamil Nadu	Introduction	table	Capital and largest city: Chennai
2	20.043	States and union territories of India	Former states and union territories > Former states	table	State: Travancore–Cochin, Capital: Trivandrum, Years:
3	19.0984	States and union territories of India	Former states and union territories > Former states	table	State: Madras State, Capital: Madras, Years: 1950–1969
4	16.2951	Tamil Nadu	Introduction	text	Tamil Nadu is the southernmost state of India. The ter
5	15.9907	Tamil Nadu	Demographics > Cities and towns	text	The capital of Chennai is the most populous urban agg

rrf

rank	score	page	section	type	text
1	0.032	Tamil Nadu	Introduction	table	Capital and largest city: Chennai
2	0.0315	Kerala	Introduction	table	Capital: Thiruvananthapuram
3	0.031	Tamil Nadu	Introduction	text	Tamil Nadu is the southernmost state of India. The tent
4	0.03	Kerala	Introduction	text	Kerala (English: /ˈkɛrələ/ ⓘ/ KERR-ə-lə; Malayalam: [ke:
5	0.0296	States and union territories of India	Former states and union territories > Former states	table	State: Travancore–Cochin, Capital: Trivandrum, Years: 1

mmr

rank	score	page	section	type	text
1	0.5393	Kerala	Introduction	table	Capital: Thiruvananthapuram
2	0.3335	Tamil Nadu	Introduction	text	Tamil Nadu is the southernmost state of India. The tenth largest Indian state by area and th
3	0.3002	Kerala	Introduction	text	Kerala (English: /ˈkɛrələ/ ⓘ/ KERR-ə-lə; Malayalam: [keːɾɐɭəm] ⓘ), officially Keralam, is a st
4	0.2809	Tamil Nadu	Introduction	table	Capital and largest city: Chennai
5	0.2695	Kerala	Introduction	table	Country: India

final

rank	score	page	section	type	text
1	0.988	Tamil Nadu	Introduction	text	Tamil Nadu is the southernmost state of India. The tenth largest Indian state by area an
2	0.9819	Kerala	Introduction	text	Kerala (English: /ˈkɛrələ/ ⓘ/ KERR-ə-lə; Malayalam: [keːɾɐɭəm] ⓘ), officially Keralam, is
3	0.9294	Kerala	Introduction	table	Capital: Thiruvananthapuram
4	0.9072	Tamil Nadu	Demographics > Cities and towns	text	The capital of Chennai is the most populous urban agglomeration in the state with mor
5	0.8043	Tamil Nadu	Introduction	table	Capital and largest city: Chennai

Figure 4. Case 1, retrieval steps for Q2 (capitals). BM25 ranks rows about former states (Travancore–Cochin, Madras State) highly because they contain "Capital". Fusion keeps the correct infobox rows near the top, and the cross-encoder's final top 5 contains only current Kerala and Tamil Nadu evidence.

10.2 Case 2: Pasted text (abbreviations, table comparison, dependent question)

Edge case: state abbreviations (UP, TN), a comparison that needs table data from two states, and a second question that depends on the first answer ("the capital of the one with more districts").

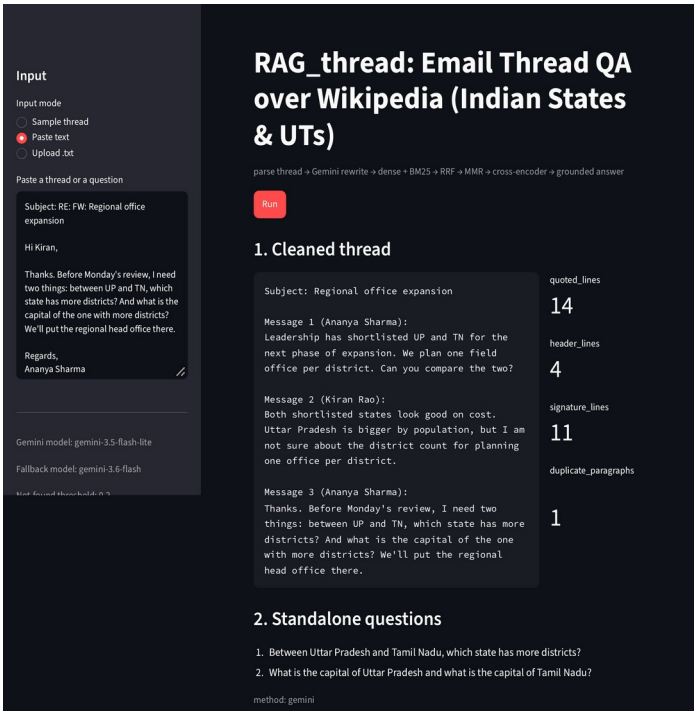


Figure 5. Case 2, input and processing: the duplicated forward is removed (duplicate_paragraphs = 1), and UP and TN are expanded to their full names in the standalone questions.

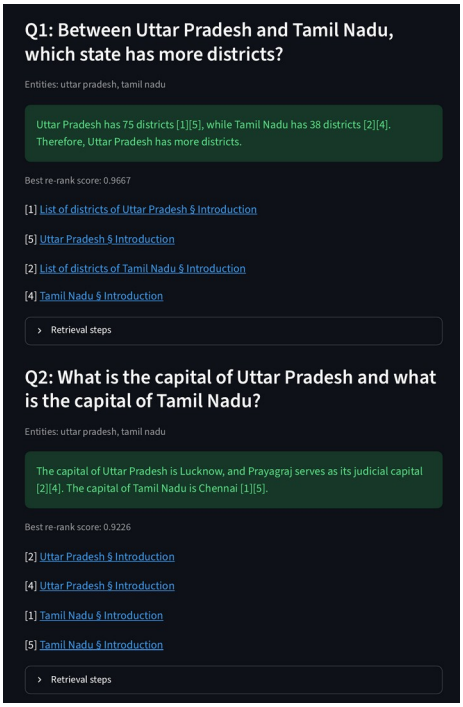


Figure 6. Case 2, output: 75 vs 38 districts, answered from the district-list pages. The dependent question is answered for both states (Lucknow, with Prayagraj as judicial capital; Chennai).

10.3 Case 3: Uploaded file (off-topic thread)

Edge case: a realistic thread whose questions (an IPL winner, cricket team size) are outside the knowledge base. Q1 scores 0.37 on sport-related chunks and passes the score gate, but the LLM gate rejects it. Q2 is rejected by the score gate (0.05).

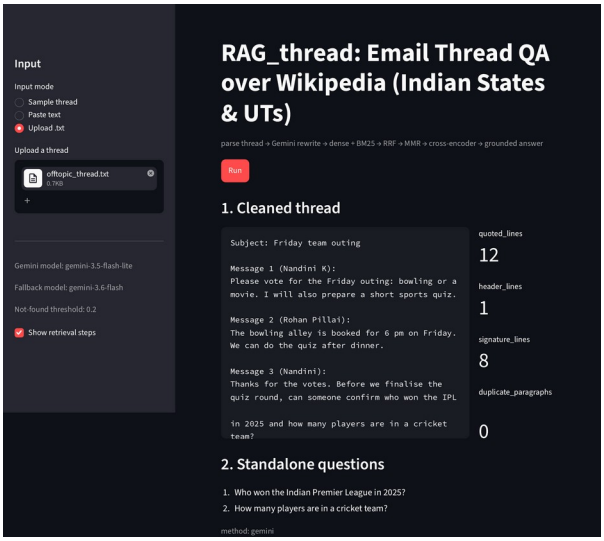


Figure 7. Case 3, input and processing: the uploaded thread is cleaned and rewritten in the same way as an on-topic thread.



Figure 8. Case 3, output: both questions return "Not found" instead of an answer from the LLM's own knowledge.

11. Limitations and Future Work

Limitation	Improvement
Paraphrases with no word overlap can fall out of the candidate pool (t07)	Query expansion or HyDE (embed an LLM-written hypothetical answer); a larger pool tuned on a separate dev set; a stronger embedding model (bge-base or e5-large) or fine-tuning on Indian-state questions
Numeric comparisons rely on the LLM reading values from rows	Load tables into pandas and answer comparison/ranking questions with exact numeric queries (text-to-pandas), using the LLM only to phrase the answer
Questions that depend on another answer are answered for all candidates ("capitals of Kerala and Tamil Nadu")	Sequential (agentic) answering: answer Q1 first, then substitute its result into Q2
Rule-based thread parsing covers common Gmail/Outlook formats only	Parse .eml/MIME headers (In-Reply-To, References) directly; learn signature detection
Rule-based entity filter (aliases must be listed)	Use the rewrite's own entity output, or fuzzy matching for misspellings
Small test set (25 threads) with keyword-based relevance labels	A larger, manually judged test set with a separate dev set for tuning τ, RRF_TOP and λ
Basic automatic faithfulness check	Claim-level verification with an NLI model or LLM-as-judge, plus human review
Wikipedia snapshot; images used only through captions	Scheduled refresh with incremental re-indexing; a vision-language model for maps and charts
Free-tier LLM rate limits	Cache rewrites/answers by thread hash, batch evaluation, or run a local LLM
Exact NumPy search	Switch to FAISS (HNSW/IVF) beyond about 10^6 chunks; the interface is unchanged

12. Conclusion

RAG_thread turns messy email threads into precise, cited answers from Wikipedia text and tables. Its main ideas are: treat the thread as memory that the LLM rewrites into standalone questions; make every table row a retrievable, citable sentence; combine lexical and semantic retrieval with rank fusion, diversity and re-ranking; and refuse to answer through two independent gates. Each design choice was checked against measurements, such as the dense miss that justified BM25 and the GDP-2040 trap that justified the LLM gate. On the test set the system reaches Recall@5 = 0.92 and 0.96 decision accuracy, with every answer it gives correct, cited and grounded.